

OAuth 2.0, JWT/JOSE, and FAPI 2.0

AI-Guided Learner Roadmap

A practical guide for building the mental model before teaching others

How to use this roadmap

Use one module at a time. For each module: draw the flow, inspect the HTTP parameters, ask AI targeted questions, then explain it back in your own words. Do not move to FAPI until OAuth flows and token validation feel natural.

Your goal	What success looks like
First	You can draw Authorization Code + PKCE from memory.
Second	You can compare OAuth flows and know when each applies.
Third	You can validate JWT/JWS tokens conceptually and explain common mistakes.
Fourth	You can explain why PAR, DPoP, mTLS, and FAPI exist.
Final	You can teach OAuth/JWT/FAPI with diagrams, examples, and threat explanations.

Version: May 2026 | Prepared as a study roadmap

Learning Roadmap at a Glance

Phase	Title	What you should master
1	OAuth Core Model	Actors, endpoints, tokens, scopes, audience, consent.
2	OAuth Flows	Authorization Code + PKCE, Client Credentials, Refresh Token, Device Code, deprecated flows.
3	OAuth Security	Redirect URI rules, state, bearer-token risk, client authentication, token validation.
4	JWT and JOSE	JWT claims, JWS signatures, JWE encryption, JWK/JWKS, key rotation.
5	Advanced OAuth	PAR, JAR, JARM, DPoP, mTLS, sender-constrained tokens.
6	FAPI 2.0	Security profile, attacker model, message signing, high-value API architecture.

Recommended learning rule

Start from flows, not token formats. JWT is only a format. OAuth flow understanding is the base. After you understand the flow, JWT/JWS/JWE become easier to place.

The Mental Model: Learn by Following the Trust

A good OAuth mental model is not memorizing terms. It is tracking trust across systems: who asks, who grants permission, who issues tokens, who validates them, and what can go wrong at each handoff.

Question	Why it matters
Who is the user?	Determines whether this is user-delegated access or machine-to-machine access.
Who is the client?	Determines whether the app can keep a secret and how it authenticates.
Who issues the token?	Defines the trusted issuer and where validation keys/metadata come from.
Who consumes the token?	Defines the token audience and scope rules.
What travels through the browser?	Browser/front-channel data is more exposed and needs protection.
What travels server-to-server?	Back-channel data can be authenticated and protected more strongly.
What can be replayed?	Leads to bearer-token risk, DPoP, mTLS, and FAPI controls.

AI learning loop for every topic

1. Ask AI to explain the concept in one paragraph.
2. Ask AI to draw the sequence diagram in text.
3. Ask AI: what HTTP endpoint is called and with which parameters?
4. Ask AI: what can go wrong here?

5. Explain it back in your own words.
6. Ask AI to quiz you with scenario questions.

1. OAuth Problem Statement

Need to know	What to observe
• OAuth exists so a client can access an API with limited permission without receiving the user password. • It supports user-delegated access and client access on its own behalf. • The access token represents permission, not the user password.	• Where does the user authenticate? • Where does the client get permission? • Where is the access token used?

Use AI like this	Definition of done
• Explain OAuth using a banking app that reads account balance. • Ask AI: Give me 5 real-world examples where OAuth is better than sharing passwords. • Ask AI to challenge you: Is this authentication or authorization?	You can explain OAuth in 2 minutes without mentioning JWT.

2. OAuth Actors and Endpoints

Need to know	What to observe
• Actors: Resource Owner, Client, Authorization Server, Resource Server. • Endpoints: authorization endpoint, token endpoint, optional introspection/revocation/metadata endpoints. • Client type matters: public clients cannot keep secrets; confidential clients can.	• Who owns the protected resource? • Which server authenticates the user? • Which server validates the access token?

Use AI like this	Definition of done
• Map the actors for: Login with Google, open banking payment, service-to-service API. • Ask AI to give you actor-mapping drills. • Draw a box diagram with all actors.	You can identify actors in any OAuth scenario.

3. Authorization Code Flow

Need to know	What to observe
• Main redirect-based flow for web apps and many user-based OAuth scenarios. • The authorization code is short-lived and exchanged at the token endpoint. • Client authentication may happen at the token	• Which request goes through the browser? • Which request is back-channel? • Where is the authorization code returned? • Where is the access token returned?

endpoint for confidential clients.	
------------------------------------	--

Use AI like this	Definition of done
• Draw the full flow with arrows. • Ask AI to simulate request/response examples. • Ask AI: what happens if the redirect URI is wrong?	You can draw the flow from memory and explain every arrow.

4. Authorization Code Flow with PKCE

Need to know	What to observe
• PKCE uses code_verifier and code_challenge to protect the code exchange. • It is especially important for public clients like mobile apps and SPAs. • It helps prevent authorization-code interception from becoming token theft.	• Where is code_challenge sent? • Where is code_verifier sent? • Why can an attacker with only the code still fail?

Use AI like this	Definition of done
• Ask AI to show normal Auth Code vs Auth Code + PKCE side by side. • Create a tiny table of code_challenge_method values. • Ask AI to quiz you with code interception scenarios.	You can explain PKCE without using complicated crypto language.

5. Client Credentials Flow

Need to know	What to observe
• Used for machine-to-machine access with no user involved. • The client authenticates directly to the token endpoint. • The resulting access token represents the client, not a user.	• Is there a browser redirect? • Is there a user? • How does the client authenticate? • What scopes should be granted?

Use AI like this	Definition of done
• Compare Auth Code vs Client Credentials. • Ask AI to build an example: backend service calling fraud service. • Ask AI: what goes wrong if the client secret leaks?	You can clearly say: Client Credentials is not user login.

6. Refresh Token Flow

Need to know	What to observe
• Access tokens should be short-lived. • Refresh tokens obtain new access tokens. • Refresh token rotation and reuse detection reduce damage from theft.	• Where is the refresh token stored? • How long does it live? • Can public clients safely store it? • What happens when reuse is detected?

Use AI like this	Definition of done
• Ask AI to show a refresh token rotation timeline. • Design storage rules for web app, mobile app, SPA, backend service. • Explain why refresh tokens are more sensitive than access tokens.	You can design basic access-token and refresh-token lifetime rules.

7. Device Authorization Flow

Need to know	What to observe
• Used when the client has limited input or cannot easily use a browser. • User receives a user_code and visits a verification URI on another device. • The client polls the token endpoint.	• Who enters the user code? • How does polling work? • What does slow_down mean? • Why is this good for CLI/TV devices?

Use AI like this	Definition of done
• Ask AI for a CLI login example. • Draw the device flow. • Compare it with Auth Code flow.	You can explain when Device Code is a better fit than normal redirects.

8. Deprecated/Legacy Flows: Implicit and Password Credentials

Need to know	What to observe
• Implicit flow and Resource Owner Password Credentials are important historical context. • They are generally avoided in modern OAuth designs. • Authorization Code + PKCE is the modern replacement for many browser/mobile cases.	• Why was implicit used before? • Why is exposing tokens in browser redirects risky? • Why should clients not collect user passwords?

Use AI like this	Definition of done
• Ask AI to compare old SPA implicit flow with modern SPA Auth Code + PKCE. • Write a one-slide warning: flows to avoid and why.	You can explain deprecated flows without recommending them.

9. Token Validation

Need to know	What to observe
• Resource servers must validate tokens before accepting API calls. • For JWT access tokens, validate issuer, audience, expiry, signature, scope, and key. • For opaque tokens, introspection may be required.	• Does this API match aud? • Is iss trusted? • Is exp still valid? • Is scope enough? • Is the signing key trusted?

Use AI like this	Definition of done
• Ask AI to give you 5 invalid JWT examples. • Practice explaining why decoding is not validation. • Create a validation checklist.	You can reject a token for the right reason.

10. JWT Basics

Need to know	What to observe
• JWT is a compact claims format. • Common claims: iss, sub, aud, exp, iat, nbf, jti, scope. • JWT does not automatically mean secure. Security depends on validation and context.	• What claims identify issuer, subject, audience, and expiry? • Is the token an access token or ID token? • Who should consume it?

Use AI like this	Definition of done
• Ask AI to annotate a JWT payload line by line. • Create a glossary of JWT claims. • Ask AI: what should not be put in a JWT?	You can explain a JWT payload without trusting it blindly.

11. JWS, JWE, and JWK/JWKS

Need to know	What to observe
• JWS provides integrity/signature protection. • JWE provides confidentiality/encryption. • JWK/JWKS represent and publish keys used for verification/encryption.	• Can others read the token? • Can others modify it? • Where does the verifier get the public key? • What does kid select?

Use AI like this	Definition of done
• Ask AI for the difference between signed and encrypted JWT using an analogy. • Trace a JWKS key rotation example. • Ask AI to quiz you on alg/kid mistakes.	You can explain: JWS proves not modified; JWE hides content.

12. OAuth Security Controls

Need to know	What to observe
• Security controls protect weak points around redirects, tokens, clients, and replay. • Important controls: state, strict redirect URI, PKCE, short token lifetime, refresh token rotation, sender-constrained tokens. • Client authentication depends on client type.	• What travels through front-channel? • Can the token be replayed? • Is the redirect exact? • Can the client keep a secret?

Use AI like this	Definition of done
• Ask AI to threat-model the Auth Code flow. • Create an attack/mitigation table. • Ask AI for scenario quizzes: what control is missing?	You can identify common OAuth mistakes and their mitigations.

13. Advanced OAuth: PAR, JAR, JARM, DPOP, mTLS

Need to know	What to observe
• PAR pushes authorization request details through a protected back-channel. • JAR signs authorization request parameters. • JARM signs authorization responses. • DPOP and mTLS sender-constrain tokens to reduce replay risk.	• Why not send everything through the browser? • What does the signature protect? • What proves possession of the key/certificate? • What changes at the resource server?

Use AI like this	Definition of done
• Ask AI to compare PAR vs JAR vs JARM. • Draw bearer token replay vs DPOP-protected token. • Ask AI to explain mTLS and DPOP in practical terms.	You can explain which advanced control solves which problem.

14. FAPI 2.0

Need to know	What to observe
• FAPI 2.0 is a high-security profile based on OAuth 2.0 and related specs. • It is intended for high-value APIs such as financial, payment, healthcare, and sensitive data APIs. • It combines strict OAuth configuration, attacker model thinking, PAR, sender-constrained tokens, and optional message signing.	• Which OAuth risks matter more for high-value APIs? • Which controls become mandatory/expected? • Where is non-repudiation required? • How would you prove compliance?

Use AI like this	Definition of done
• Ask AI to design a secure open banking flow. • Map FAPI controls to attacks. • Ask AI to review your architecture and point out missing controls.	You can describe FAPI as stricter OAuth, not a separate replacement protocol.

OAuth Flow Comparison Cheat Sheet

Flow	User?	Browser redirect?	Token endpoint?	Main use case	Today?
Authorization Code + PKCE	Yes	Yes	Yes	Web, mobile, SPA, user consent	Recommended
Client Credentials	No	No	Yes	Service-to-service APIs	Recommended
Refresh Token	Sometimes	No	Yes	Renew access token	Recommended with controls
Device Authorization	Yes	Partial	Yes	TV, CLI, limited-input devices	Recommended when appropriate
Resource Owner	Yes	No	Yes	Legacy trusted	Avoid

Password Credentials				apps	
Implicit	Yes	Yes	No token endpoint exchange	Legacy browser apps	Avoid

Suggested 30-Day Study Plan

Days	Focus	Output you create
1-3	OAuth actors, endpoints, tokens, scopes, audience, consent	One-page OAuth mental model diagram
4-7	Authorization Code Flow and PKCE	Sequence diagram plus request/response notes
8-10	Client Credentials, Refresh Token, Device Code	Flow comparison table
11-14	Deprecated flows and OAuth mistakes	Avoid/recommend decision tree
15-18	Token validation and JWT claims	JWT validation checklist
19-21	JWS, JWE, JWKS and key rotation	Token trust checklist
22-25	OAuth security controls: state, redirect, replay, client auth	Attack/mitigation matrix
26-28	PAR, DPoP, mTLS, JAR/JARM	Advanced controls map
29-30	FAPI 2.0 overview	High-value API architecture diagram

AI Prompt Bank

Understand a concept

- Explain [topic] in plain language, then in technical language.
- Give me a sequence diagram for [flow] using Client, Browser, Authorization Server, and Resource Server.
- Show me the HTTP request parameters for each step of [flow].

Test understanding

- Quiz me with 10 scenario questions about [topic]. Wait for my answer before giving feedback.
- Give me 5 wrong explanations of [topic] and ask me to fix them.
- Ask me to choose the correct OAuth flow for different application types.

Threat model

- Threat-model this OAuth flow and list attacks, impact, and mitigations.
- What can go wrong if the redirect URI is not exact?

- How would an attacker replay a bearer token, and how do DPoP/mTLS help?

Prepare to teach

- Turn this topic into a 10-minute mini-lesson with diagram, analogy, and quiz.
- What are the top 5 misconceptions learners have about [topic]?
- Review my explanation and point out missing or incorrect parts.

Readiness Checklist Before You Teach

Area	You are ready when...
OAuth flows	You can draw Auth Code + PKCE and Client Credentials without notes.
Tokens	You can explain code vs access token vs refresh token vs ID token.
JWT/JWS/JWE	You can explain format vs signature vs encryption.
Validation	You can list issuer, audience, expiry, signature, scope, and key checks.
Security	You can connect attacks to mitigations: PKCE, state, redirect matching, DPoP/mTLS.
FAPI	You can explain FAPI as strict OAuth for high-value APIs.
Teaching	You can answer: what problem does this solve, when to use it, and what can go wrong?

Official Resource Roadmap

Topic	Primary resources to read
OAuth 2.0 core	RFC 6749 - OAuth 2.0 Authorization Framework; RFC 6750 - Bearer Token Usage
PKCE	RFC 7636 - Proof Key for Code Exchange
OAuth security	RFC 9700 - Best Current Practice for OAuth 2.0 Security
JWT/JOSE	RFC 7519 JWT; RFC 7515 JWS; RFC 7516 JWE; RFC 7517 JWK; RFC 7518 JWA
JWT access tokens	RFC 9068 - JWT Profile for OAuth 2.0 Access Tokens
Token operations	RFC 7662 Token Introspection; RFC 7009 Token Revocation; RFC 8414 Authorization Server Metadata
Advanced OAuth	RFC 9126 PAR; RFC 9101 JAR; RFC 9449 DPoP; RFC 8705 mTLS
OpenID Connect	OpenID Connect Core 1.0 - for ID tokens and authentication layer
FAPI 2.0	OpenID Foundation FAPI 2.0 Security Profile, Attacker Model, Message Signing

How to read specs without getting stuck

Do not read every RFC front-to-back first. For each topic, read: abstract, terminology, protocol flow, security

considerations, then examples. Use AI to summarize one section at a time and ask it to produce diagrams and test questions.